

Kafka Log Compaction

- ❖ Like Cassandra, LevelDB, RocksDB, and others, Kafka uses long sequential disk access for read and writes
- ❖ Kafka uses tombstones instead of deleting records right away
- ❖ Modern Disks have somewhat unlimited space and are fast
- ❖ Kafka can provide features not usually found in a messaging system like holding on to old messages for a really long time
 - ❖ This flexibility allows for interesting application of Kafka

- ❖ Kafka cluster retains all published records
 - ❖ Time based – configurable retention period
 - ❖ Size based - configurable based on size
 - ❖ Compaction - keeps latest record
- ❖ Kafka uses Topic ***Partitions***
- ❖ Partitions are broken down into ***Segment*** files

NAME	DESCRIPTION
log.dir	topic logs will be stored in this or log.dirs
log.dirs	The directories where the Topics logs are kept used for JBOD
log.flush.interval.messages	Accumulated messages
log.flush.interval.ms	Maximum time that a topic message is kept in memory before flushed to disk. If not set uses log.flush.scheduler.interval.ms
log.flush.scheduler.interval.ms	Interval that topic messages are periodically flushed from memory to log.

log.retention.bytes	Delete log records by size. The maximum size of the log before deleting its older records	long	-1
Log.retention.hours	Delete log records by time hours. Hours to keep a log file before deleting older records(in hours),tertiary to log.retention.ms property.	int	168
log.retention.minutes	Delete log records by time minutes. Minutes to keep a log file before deleting it,secondary to log.retention.ms property. If not set,use log.retention.hours is used	Int	Null
log.retention.ms	Delete log records by time milliseconds. Milliseconds to keep a log file before deleting it, If not set, use log.retention.minutes.	Long	Null

NAME	DESCRIPTION	TYPE	DEFAULT
Log.roll.hours	Time period before rolling a new topic log segment.(secondary to log.roll.ms property)	Int	1
log.roll.ms	Time period in milliseconds before rolling a new log segment. If not,uses log.roll.hours.	Long	
log.segment.bytes	The maximum size of a single log segment file.	Int	1,073,741,82
log.segment.delete.delay.ms	Time period to wait before deleting a segment file from the filesystem	Long	60,000

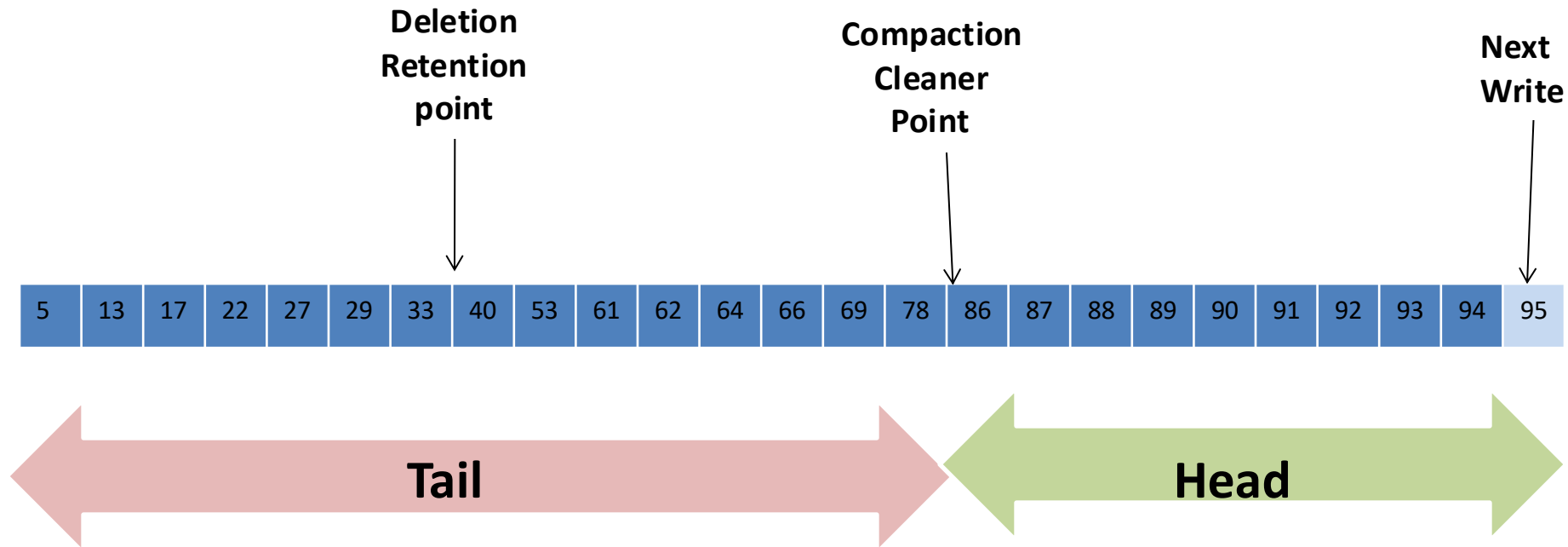
- ❖ Recall Kafka can delete older records based on
 - ❖ time period
 - ❖ size of a log
- ❖ Kafka also supports log compaction for record key compaction
- ❖ Log compaction: keep latest version of record and delete older versions

- ❖ Log compaction retains last known value for each record key
- ❖ Useful for restoring state after a crash or system failure, e.g., in- memory service, persistent data store, reloading a cache
- ❖ Data streams is to log changes to keyed, mutable data,
 - ❖ e.g., changes to a database table, changes to object in in-memory microservice
- ❖ Topic log has full snapshot of final values for every key - not just recently changed keys
- ❖ Downstream consumers can restore state from a log compacted topic

- ❖ Log has head and tail
- ❖ Head of compacted log identical to a traditional Kafka log
- ❖ New records get appended to the head :
 - ❖ the active segment is never cleaned by this process
- ❖ Log compaction works at tail of the log
- ❖ Tail gets compacted
- ❖ Records in tail of log retain their original offset when written after compaction

Compaction Tail/Head

Tos



- ❖ All offsets remain valid, even if record at offset has been compacted away (next highest offset)
- ❖ Compaction also allows for deletes. A message with a key and a null payload acts like a tombstone (a delete marker for that key)
 - ❖ Tombstones get cleared after a period.
- ❖ Log compaction periodically runs in background by recopying log segments.
- ❖ Compaction does not block reads and can be throttled to avoid impacting I/O of producers and consumers

Before Compaction

Offset	13	17	19	20	21	22	23	24	25	26	27	28
Keys	K1	K5	K2	K7	K8	K4	K1	K1	K1	K9	K8	K2
Values	V5	V2	V7	V1	V4	V6	V1	V2	V9	V6	V22	V25

Cleaning

Only keeps latest version of key. Older duplicates not needed.

Offset	17	20	22	25	26	27	28
Keys	K5	K7	K4	K1	K9	K8	K2
Values	V2	V1	V6	V9	V6	V22	V25

After Compaction

- ❖ If consumer stays caught up to head of the log, it sees every record that is written.
 - ❖ Topic config ***min.compaction.lag.ms*** used to guarantee minimum period that must pass before message can be compacted.
- ❖ Consumer sees all tombstones as long as the consumer reaches head of log in a period less than the topic config ***delete.retention.ms*** (the default is 24 hours).
- ❖ Compaction will never re-order messages, just remove some.
- ❖ Offset for a message never changes.
- ❖ Any consumer reading from start of the log, sees at least final state of all records in order they were written

- ❖ Log cleaner does log compaction.
 - ❖ Has a pool of background compaction threads that recopy log segments, removing records whose key appears in head of log
- ❖ Each compaction thread works as follows:
 - ❖ Chooses topic log that has highest ratio: log head to log tail
 - ❖ Recopies log from start to end removes records whose keys occur later
- ❖ As log partition segments cleaned, they get swapped into log partition
 - ❖ Additional disk space required: only one log partition segment
 - ❖ not whole partition

- ❖ To turn on compaction for a topic
 - ❖ topic config ***log.cleanup.policy=compact***
- ❖ To start compacting records after they are written
 - ❖ topic config ***log.cleaner.min.compaction.lag.ms***
 - ❖ Records wont be compacted until after this period

Broker Config for Log Compaction

Tos

NAME	DESCRIPTION	TYPE	DEFAULT
log.cleaner.backoff.ms	Sleep period when no logs need cleaning	Long	15,000
log.cleaner.dedupe.buffer.size	The total memory for log dedupe process for all cleaner threads	Long	134,217,728
log.cleaner.delete.retention.ms	How long record delete markers (tombstones) are retained	Long	86,400,000
log.cleaner.io.buffer.size	Total memory used for log cleaner I/O buffers for all cleaner threads	Int	524,288
log.cleaner.io.max.bytes.per.second	This is a way to throttle the log cleaner if it is taking up too much time	Double	1.7976931348623157E308
log.cleaner.min.cleanable.ratio	The minimum ratio of dirty head log to total log(head and tail) for a log to get selected for cleaning	Double	0.5
log.cleaner.min.compaction.lag.ms	Minimum time period a new message will remain uncompactd in the log.	Long	0
log.cleaner.threads	Threads count used for log cleaning. Increase this if you have a lot of log compaction going on across many topic log partitions	Int	1
log.cleanup.policy	The default cleanup policy for segment files that are beyond their retention window. Valid policies are:"delete" and "compact". You could use log compaction just for older segment files instead of deleting them, you could just compact them.		

- ❖ Adding Kafka Brokers to cluster is simple
 - ❖ need unique broker id
 - ❖ new Kafka Brokers are not automatically assigned Topic partitions
 - ❖ You need to migrate partitions to it
- ❖ Migrating Topic Partitions is manually initiated
 - ❖ New Kafka Broker becomes followers of partitions
 - ❖ When it becomes ISR set member, then it gains leadership over partitions assigned to it
 - ❖ Once it becomes leader, existing replica will delete partition data if needed
- ❖ Kafka provides a partition reassignment tool

- ❖ partition can be moved across brokers
- ❖ avoid hotspots, balance load on brokers
- ❖ you have to look at load on Kafka Broker
 - ❖ use ***kafka-consumer-groups.sh***
 - ❖ other admin tools to find hotspots (top, KPIs, etc.)
 - ❖ balance as needed

- ❖ **GENERATE A PLAN—*generate***
 - ❖ Inputs: Topics List, and Kafka Broker List
 - ❖ Generates ***reassignment plan*** to move all topic partitions to new Kafka Brokers
- ❖ **EXECUTE A PLAN —*execute***
 - ❖ Input: ***reassignment plan*** (***--reassignment-json-file***)
 - ❖ Action: Does partition reassignment using plan
- ❖ **CHECK STATUS OF EXECUTE PLAN —*verify***
 - ❖ Shows status of —*execute*
 - ❖ Outputs: Completed Successfully, Failed or In-Progress

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-reassign-partitions.sh --bootstrap-server kafka0:9092 --reassignment-json-file custom-reassignment.json --execute
Current partition replica assignment

{"version":1,"partitions":[{"topic":"my-kafka-topic","partition":12,"replicas":[1,2,3],"log_dirs":["any","any","any"]}]}

Save this to use as the --reassignment-json-file option during rollback
Successfully started partition reassignment for my-kafka-topic-12
[root@kafka0 scripts]#
```

Create a file named custom-reassignment.json with the following content:

```
{"version":1,"partitions":[{"topic":"my-kafka-topic","partition":12,"replicas":[1,2,3]}]}
```

- ❖ Added 4th Broker(id - 3)! Now we want it to have some partitions
- ❖ **custom-reassignment.json** - list of topics to move in JSON format
- ❖ Generates assignment plan which needs to be edited

Generated Partition Assignment Plan

Tos



```
1 {  
2   "version": 1,  
3   "partitions": [  
4     {  
5       "topic": "stocks",  
6       "partition": 7,  
7       "replicas": [  
8         0,  
9         1,  
10        2  
11      ]  
12    },  
13    {  
14      "topic": "stocks",  
15      "partition": 2,  
16      "replicas": [  
17        3,  
18        2,  
19        0  
20      ]  
13  ]  
14 }
```

- ❖ Assignment Plan
- ❖ List of partitions
- ❖ List of Replicas
- ❖ Replicas might be removed to new Kafka Broker after plan executes
- ❖ Need to execute plan

- ❖ After we add a new broker,
 - ❖ add it to the `—broker-list`
 - ❖ Run generate plan
 - ❖ Execute plan
- ❖ To decommission Kafka Broker
 - ❖ Remove it from the `—broker-list`
 - ❖ Run generate plan, execute generate plan

```
1 #!/usr/bin/env bash
2 CONFIG=`pwd`/config
3 cd ~/kafka-training
4
5 # Execute reassignment plan
6 kafka/bin/kafka-reassign-partitions.sh --execute \
7     --reassignment-json-file "$CONFIG/assignment-plan.json" \
8     --throttle 100000 \
```

- ❖ Executes reassignment plan
- ❖ Use generated plan or use modified generated plan
- ❖ Set throttle rate (optional) so it does not all happen at once
 - ❖ reduces load on Kafka Brokers

```
[root@kafka0 scripts]#  
[root@kafka0 scripts]# /opt/kafka/bin/kafka-reassign-partitions.sh --bootstrap-server kafka0:9092 --reassignment-json-file custom-reassignment.json --verify  
Status of partition reassignment:  
Reassignment of partition my-kafka-topic-12 is complete.  
  
Clearing broker-level throttles on brokers 0,1,2,3  
Clearing topic-level throttles on topic my-kafka-topic  
[root@kafka0 scripts]# █
```

- ❖ Verify/Monitor reassignment plan
- ❖ Use generated plan or use modified generated plan that is already running
- ❖ Let's you know when the plan is done

Generate Partition Reassignment Plan

Tos

```
[root@kafka0 scripts]# /opt/kafka/bin/kafka-reassign-partitions.sh --bootstrap-server kafka0:9092 --broker-list 0,1,2 --topics-to-move-json-file topic.json --generate
Current partition replica assignment
{"version":1,"partitions":[{"topic":"my-kafka-topic","partition":0,"replicas":[2,0,1],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":1,"replicas":[1,2,0],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":2,"replicas":[0,1,2],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":3,"replicas":[2,1,0],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":4,"replicas":[1,0,2],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":5,"replicas":[0,2,1],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":6,"replicas":[2,0,1],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":7,"replicas":[1,2,0],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":8,"replicas":[0,1,2],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":9,"replicas":[2,1,0],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":10,"replicas":[1,0,2],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":11,"replicas":[0,2,1],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":12,"replicas":[1,2,3],"log_dirs":["any","any","any"]}]}

Proposed partition reassignment configuration
{"version":1,"partitions":[{"topic":"my-kafka-topic","partition":0,"replicas":[2,1,0],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":1,"replicas":[0,2,1],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":2,"replicas":[1,0,2],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":3,"replicas":[2,0,1],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":4,"replicas":[0,1,2],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":5,"replicas":[1,2,0],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":6,"replicas":[2,1,0],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":7,"replicas":[0,2,1],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":8,"replicas":[1,0,2],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":9,"replicas":[2,0,1],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":10,"replicas":[0,1,2],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":11,"replicas":[1,2,0],"log_dirs":["any","any","any"]}, {"topic":"my-kafka-topic","partition":12,"replicas":[2,1,0],"log_dirs":["any","any","any"]}]}

[root@kafka0 scripts]#
```

topic.json

```
{
  "topics": [
    {
      "topic": "my-kafka-topic"
    }
  ],
  "version": 1
}
```

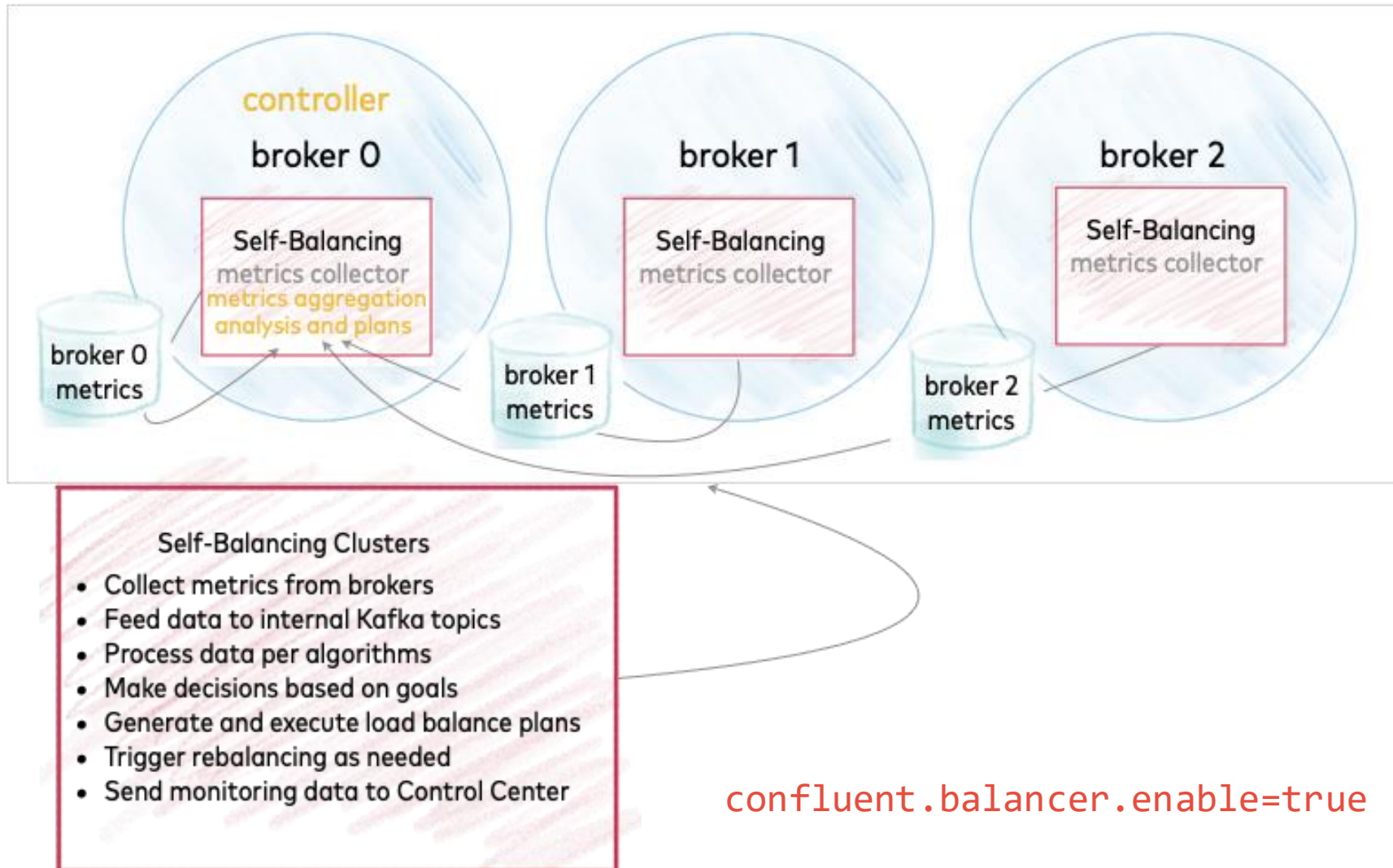
- ❖ Remove 4th Broker (3)! Now we want it reassign its partitions
- ❖ Generates assignment plan that moves partitions to 0,1,2

Self-Balancing Clusters is a Confluent feature that is designed to simplify the process of managing an Apache Kafka® cluster. With this feature enabled, a cluster automatically rebalances partitions across brokers when new brokers are added or existing brokers are removed. This ensures that data is evenly distributed across the cluster, which can improve performance and reduce the risk of data loss in the event of a broker failure.

Self-Balancing offers the following advantages over Auto Data Balancer

- Cluster balance is continuously monitored so that rebalances run whenever they are needed
- Broker failure conditions are detected and addressed automatically
- No additional tools to run (built into the brokers)
- Works with Confluent Control Center and offers a REST API
- Much faster rebalancing (several orders of magnitude)

Architecture of a Self-Balancing cluster



- The incoming and outgoing byte rate for reassignments is tracked by `kafka.server:type=BrokerTopicMetrics,name=ReassignmentBytesInPerSec` and `kafka.server:type=BrokerTopicMetrics,name=ReassignmentBytesOutPerSec`. These metrics are reported by each broker.
- The number of pending and in-progress reassignment tasks currently tracked by Self-Balancing are tracked by `kafka.databalancer:type=Executor,name=replica-action-pending` and `kafka.databalancer:type=Executor,name=replica-action-in-progress`. These metrics are reported from the broker with the active data balancer instance (the controller).
- The maximum follower lag on each broker is tracked by `kafka.server:type=ReplicaFetcherManager,name=MaxLag,clientId=Replica`. Reassigning partitions will cause this metric to jump to a value corresponding to the size of the partition and then slowly decay as the reassignment progresses. If this value slowly rises rather than slowly falling over time, the replication throttle is too low.

Lab :

Understanding Log Architecture and Compaction – 60 Minutes.

Confluent:

Self-Balancing Clusters (60 Minutes)

Apache Kafka:

Movement of Partition from a Broker to another Broker-60 Minutes.